

Contents

1 Problem statement

1

1 Problem statement

The goal is a Chapel program that computes the Euclidean minimum $M(K)$ of a number field K , for arbitrary degree, using Pierre Lezowski’s algorithm (`src/.c`) as the mathematical reference, but doing the numeric heavy-lifting in floating point/Chapel instead of Pari, with Pari used only for one-time field setup (already done by `generate_field.gp`). The current Chapel attempts (`firstCut.chpl` ... `sixthCut.chpl`, `kernel.chpl`, `kernel.py`, `exactMin.chpl`) frequently return wrong minima and are hard-capped at degree 4. ## Diagnosis of the current code Reading `chplPlan.org` and `note.txt` alongside the actual Chapel sources shows the previous implementation attempts did **not** port Lezowski’s algorithm — they invented an unrelated “AMR box/disk sieve” that is unsound: / `kernel.chpl:54` (`testBoxCovering`) only checks $N(x) < \text{bound}$ and $N(\text{unit} \cdot x) < \text{bound}$. Since every unit has $|N(\text{unit})| = 1$, the “unit” check is numerically almost the same test as the first one — it does nothing useful. Real absorption requires testing $N(x -) < \text{bound}$ against many candidate integers , which never happens here (only $= 0$ is ever tried). * `sixthCut.chpl:68` patches this by brute-force scanning integer offsets $\{-1, 0, 1\}$ per coordinate and unit exponents $\{-2..2\}$ (hard-coded up to 3 coefficients and 2 units — this is the literal source of the “degree 4” ceiling). This is an ad hoc heuristic, not derived from the field’s geometry, so it silently misses the real absorbing elements for larger fields/discriminants/rank 2 unit groups — exactly the failure the user already reproduced for x^3-3x-1 in `note.txt`. * `exactMin.chpl:38,45` calls `kernel.testBoxCovering(currentBox, ...)` where `currentBox` is a `Box` record, but `kernel.chpl`’s `testBoxCovering` expects a plain `[] real(64)` — these files are already mutually inconsistent. * There is no equivalent anywhere of `small_elts` (`src/main.c:130`, enumerates candidate algebraic integers bounding a given norm target) or of the real absorption formula `norme_bricolee` (`src/basef.c:439`). These are the mathematical core of Lezowski’s method, and both are missing entirely. * No unit-recentering (`vecteurs_possibles2/=test_des_unites`, `src/main.c:676,757`), no problem graph/cycle detection (Tarjan port, `src/graph.c:58,102`), and no exact finalization via Pari (`calcul_min_liste_cycles`, `src/pari_min_c.c:675`) exist in Chapel — so even where the sieve reports “uncleared”, there is no

way to certify a matching lower bound, which is why runs end in “uncertain” territory (`note.txt`). **Conclusion: this needs a rewrite of the sieve/kernel, not a patch.** The good news is the parts that are salvageable are salvageable: the `field_data.txt` format, `generate_field.gp`, and the `box`→embedding projection math already in `sixthCut.chpl` (`computeBoxProjections`≠`calculateBo` lines 30-65) are structurally correct primitives — they were just never wired to a correct absorption test. `##` Key algorithmic facts established during research * Degree n , signature $(r1,r2)$ always satisfy $n = r1+2\cdot r2$; boxes are naturally described in *integer-coefficient space* (dimension n), then projected into the $r1+r2$ real/complex embedding slots for the norm bound — this is exactly what the current complex-compact `field_data.txt` format and `computeBoxProjections` already do, so **no changes to `generate_field.gp`≠`field_data.txt` are needed for the corrected core sieve.** * Lezowski’s absorption test (`test_absorption`≠`norme_bricolee`) is sound using **only** the enumerated small-integer candidate list per target bound K — the fundamental-unit action (`test_des_unites`) is an *acceleration and exact-certification* device, not a soundness requirement for the basic covering test. This means a first, fully correct implementation can skip unit-recentering and the cycle/graph machinery entirely and still be mathematically rigorous for proving upper bounds. * A sieve that proves “all boxes absorbed for bound K ” only ever certifies $M(K) < K$ (a numeric upper bound). It can never certify a matching lower bound by itself — Lezowski’s C code gets exact answers via the unit-orbit cycle decomposition + an exact Pari evaluation at the cycle’s rational critical point. We can get the same certification much more simply: take the box(es) that most resist absorption at maximum sieve depth (rational coordinates), and make **one exact Pari call** to compute `min_{x-} N(x-)` there exactly. If that exact value matches the numeric upper bound from the sieve, the minimum is proved exactly, without porting Tarjan/graph-simplification at all. * Chapel domains must have a compile-time-constant rank, so the arbitrary-degree candidate-integer enumeration (`small_elts`) cannot use a rank- n domain; it must use a flattened 1-D index space over $(\text{limsup}-\text{liminf}+1)^{\text{degree}}$ decoded via mixed-radix arithmetic (generalizing the C code’s manual odometer-style increment in `main.c:150-232`), so it parallelizes cleanly with a single `forall`. * Environment check: `gp` (Pari/GP), `gcc`, and `chpl` (2.9.0) are all installed locally, and the existing `Makefile`≠`src/*.c` can build the reference `euclid` C binary — this gives a ground-truth oracle for validation without re-deriving expected minima by hand. `#` Proposed approach: staged rewrite `##` Phase 1 — Correct, arbitrary-degree decision sieve (numeric upper bounds) New Chapel modules (replacing the box-sieve logic in `sixthCut.chpl`≠`exactMin.chpl`; the

field-file parser in `kernel.chpl` is reused/cleaned up): * `NumberField.chpl`: parses `field_data.txt` into a record (degree, r1, r2, basis embeddings, unit embeddings) — cleanup of `kernel.chpl`’s loader. * `SmallElements.chpl`: implements the arbitrary-degree analogue of `small_elts` (`main.c:130`) — computes the per-coordinate bound from row-sums of the embedding matrix and $K^{\{1/\text{degree}\}}$, then enumerates all integer coefficient vectors in that box via the flattened mixed-radix `forall` described above, keeping only those whose embedding actually falls in range (mirrors `main.c:155-232`). * `Sieve.chpl`: keeps the `Box` record and `computeBoxProjections`/`=calculateBoxMaxNorm` primitives (ported from `sixthCut.chpl:10-65`), and adds the real absorption test: for a box, iterate over the current `SmallElements` list, subtract each candidate’s embedding from the box’s projected center, and check `calculateBoxMaxNorm(shifted) < K` (the correct generalization of `norme_bricolee`, `basef.c:439`) — short-circuiting on the first success and moving that candidate to the front of the list for locality (mirrors `ramene_en_tete3`, `basef.c:500`). Boxes failing absorption are bisected into 2^{degree} children (this part of the existing AMR code, `sixthCut.chpl:241-260`, is structurally fine and is kept), tracked with a `list`/`=array` per depth level and processed with `=forall`. * A driver that runs `cut+absorb` to a resolution limit for a given K , reports success/failure, and performs bisection search over K , replacing the naive `epsilon`-only binary search in `sixthCut.chpl:273-301` with the same “shrink K based on evidence, keep best success” strategy as `main.c`’s `main_loop`/`=outer` `=while` (`main.c:2682` onward) generalized to arbitrary degree. This phase alone fixes both reported defects: it is mathematically sound (real absorption against real candidates) and has no dimension-specific hard-coding, so it works for arbitrary degree. ## Phase 2 — Unit-action acceleration (optional but recommended before scaling up) * Extend `NumberField.chpl` to compute the inverse embedding matrix (`sigma_inv`) via Chapel’s `LinearAlgebra` module (unpack the complex-compact embedding matrix to a real $n \times n$ matrix, invert, done once at load time — no `generate_field.gp` changes needed). * Port `vecteurs_possibles2`/`=test_des_unites` (`main.c:676-897`): apply each fundamental unit to a surviving box’s embeddings, use `sigma_inv` to re-round the twisted box back toward the origin in coefficient space, and re-test absorption. This mirrors the real acceleration loop (`boucle_unites_et_decoupage_v4`, `main.c:951`) and should sharply cut down on the bisection depth needed for fields with larger regulators, without which some fields may be numerically slow (not wrong, just slow) under Phase 1 alone. ## Phase 3 — Exact certification (targeted, simpler than the C tool’s graph/cycle machinery) * `Certify.chpl`: once the Phase 1/2 sieve stalls at maximum depth for a shrinking K , collect the most-resistant

box center(s) (rational coordinates from the grid), and shell out **once** to **gp** (Chapel **Subprocess** module) with a small script that computes **min_{small} N(x-)** exactly at that rational point using Pari’s exact arithmetic (reusing the same bounding-box logic as **SmallElements**, just done exactly instead of in float). * Compare the exact value to the numeric upper bound from the sieve: if they match (typically true for the well-known small examples — e.g. $x^3-3x-1 \rightarrow 1/3$), report the proven exact Euclidean minimum. This sidesteps porting Tarjan (**graph.c:58**) and the problem-graph simplification (**main.c:1345-1843**) entirely. * If this simplified certification is ever insufficient (fields with genuinely non-isolated resistant orbits under rank 2 unit groups), the full cycle-decomposition port remains a documented fallback but is out of scope unless Phase 3’s simpler approach proves inadequate in testing. ## Parallelization roadmap (kept in mind from Phase 1 onward, not built until correctness is solid) * Single-locale multi-core: **forall** over the small-elements enumeration and over each level’s box list (already the pattern in the existing code — kept). * Multi-locale: shard the per-level “problems” list across locales (e.g. **Block**-distributed array or a per-locale list merged with an all-to-all at the end of each depth level), since each box’s absorption test is independent given the (replicated, typically small) candidate list. * GPU: **computeBoxProjections** $\neq$ **calculateBoxMaxNorm** $\neq$ **absorption-against-candidate-list** is pure flat numeric work with no dynamic allocation in the inner loop --- a natural **foreach** $\neq$ **@assertOnGpu** target once box/candidate data is restructured into flat arrays-of-structs-of-arrays instead of **list(Box)**. # Validation strategy * Build the reference **src C euclid** tool (**make** in **euclid-1.0/**, pari + gcc already present) to generate ground-truth minima for a battery of fields spanning degree/signature variety: the existing saved fixtures **q2.txt** (x^2), **cc23.txt** (x^3+x^2+1 , disc 23), plus x^3-3x-1 (rank-2 unit group, known min $1/3$, the case that broke the old sieve), x^2-61 , and at least one quintic/sextic example to prove the degree generalization. * Add a small test harness script that runs **generate_field.gp** (already present) then the new Chapel pipeline for each fixture and diffs against the C tool’s reported minimum within tolerance. * Move **firstCut.chpl** ... **sixthCut.chpl**, **exactMin.chpl**, **kernel.py**, **tmp.chpl** into an **attic/** sub-directory rather than deleting them, since they’re superseded but harmless as history.